Pandas 2.0 and the Arrow revolution (datapythonista.me)

314 points by ZeroCool2u 8 months ago | hide | past | favorite | 93 comments

MrPowers 8 months ago | next [–]

The Arrow revolution is particularly important for pandas users.

pandas DataFrames are persisted in memory. The rule of thumb was for RAM capacity / dataset size in memory to be 5-10x as of 2017. Let's assume that pandas has made improvements and it's more like 2x now.

That means you can process datasets that take up 8GB of RAM in memory on a 16GB machine. But 8GB of RAM in memory is a lot different than what you'd expect with pandas.

pandas historically persisted string columns as objects, which was wildly inefficient. The new string[pyarrow] column type is around 3.5 times more efficient from what I've seen.

Let's say a pandas user can only process a string dataset that has 2GB of data on disk (8GB in memory) on their 16GB machine for a particular analysis. If their dataset grows to 3GB, then the analysis errors out with an out of memory exception.

Perhaps this user can now start processing string datasets up to 7GB (3.5 times bigger) with this more efficient string column type. This is a big deal for a lot of pandas users.

ingenieroariel 8 months ago | parent | next [–]

To your point: If you have your data as parquet files and load it into memory using DuckDB before passing the arrow object to Pandas you can run in arbitrary large datasets as long as you aggregate them or filter them somehow.

In my case I run analysis on an Apple M1 with 16GB of RAM and my files on disk are thousands of parquet files that add up to hundreds of gigs.

Apart from that: Being able to go from duckdb to pandas very quickly to make operations that make more sense on the other end and come back while not having to change the format is super powerful. The author talks about this with Polars as an example.

IanCal 8 months ago | root | parent | next [–]

> Being able to go from duckdb to pandas very quickly to make operations that make more sense on the other end and come back while not having to change the format is super powerful

I can't stress enough how much I think this is truly transformative. It's generally nice as a working pattern, but much more importantly it lets the *scale* of a problem that a tool needs to solve shrink dramatically. Pandas doesn't need to do everything, nor does DuckDB, nor does some niche tool designed to perform very specific forecasting - any can be slotted into an in memory set of processes with no overhead. This lowers the barrier to entry for new tools, so they should be quicker and easier to write for people with detailed knowledge just in their area.

It extends beyond this too, as you can then also get free data serialisation. I can read data from a file with duckdb, make a remote gRPC call to a flight endpoint written in a few lines of python that performs whatever on arrow data, returns arrow data that gets fed into something else... in a very easy fashion. I'm sure there's bumps and leaky abstractions if you do deep work here but I've absolutely got remote querying of dataframes & files working with a few lines of code, calling DuckDB on my local machine through ngrok from a colab instance.

MrPowers 8 months ago | root | parent | prev | next [–]

Yes, I agree with your assessment that technologies that can deal with larger than memory datasets (e.g. Polars) can be used to filter data so there are less rows for technologies that can only handle datasets a fraction of the data (e.g. pandas).

Another way to solve this problem is using a Lakehouse storage format like Delta Lake so you only read in a fraction of the data to the pandas DataFrame (disclosure: I am on the Delta Lake team). I've blogged about this and think predicate pushdown filtering / Z ORDERING data is more straightforward that adding an entire new tech like Polars / DuckDB to the stack.

If you're using Polars of course, it's probably best to just keep on using it rather than switching to pandas. I suppose there are some instances when you need to switch to pandas (perhaps to access a library), but think it'll be better to just stick with the more efficient tech normally.

mr337 8 months ago | root | parent | prev | next [–]

This sounds very neat, time to look into DuckDB.

taeric 8 months ago | parent | prev | next [–]

The memory expectations for so many programmers going into pandas baffle me. In particular, noone was batting an eye that a 700 meg CSV file would take gigs to hold in memory. Just convincing them to specify the dtypes and to use categorical where appropriate has had over 70% reductions on much of our memory requirements. Not shockingly, they go faster, too.

If there are efforts to help this be even better, I heartily welcome them.

__mharrison__ 8 months ago | root | parent | next [–]

When I'm teaching Pandas, the first thing we do after loading the data is inspect the types. Especially if the data is coming from a CSV. A few tricks can save 90+% of the memory usage for categorical data.

This should be a step in the right direction, but it will probably still require manually specifying types for CSVs.

taeric 8 months ago | root | parent | next [–]

Yeah, I expect most efforts to just help make the pain not as painful. And specifying the data types is not some impossible task and can also help with other things.

tootie 8 months ago | parent | prev | next [–]

Isn't this also addressed by using Dask?

MrPowers 8 months ago | root | parent | next [–]

Dask sidesteps the problem by splitting up the data into smaller pandas DataFrames and processing them individually, rather than all at once. But the fundamental issue of pandas storing strings as objects in DataFrames (which is really inefficient) is also problematic with Dask (because a Dask DataFrame is composed of a lot of smaller pandas DataFrames). The technologies that really address this problem are Polars/PySpark - technologies that are a lot more memory efficient.

0cf8612b2e1e 8 months ago | root | parent | prev | next [–]

I am always tripping over my feet when I use Dask. Definitely due to inexperience (I only reach for it when my usual Pandas tricks fail me), but I never have a good time there. Pandas idiom X will still blow up memory, you have to know Dask idiom Y, etc.

I am glad Dask exists, but it is not a seamless solution where you can just swap in a Dask dataframe.

TrapLord_Rhodo 8 months ago | root | parent | prev | next [–]

Dask can utilize pandas dataframes and make them run across different cores. Also is great for tasks that take up too much memory as discussed by the OP. This may replace the need for you to use DASK in alot of projects since you can work with 3.5x the RAM now.

jwmoz 7 months ago | root | parent | prev | next [–]

I tried dask specifically for this task, processing a larger file on an 8GB mem macbook air. It was wildly ineffective. Ended up having to chop up the data and do it myself.

yamrzou 8 months ago | prev | next [–]

The submission link points to the blog instead of the specific post. It should be: https://datapythonista.me/blog/pandas-20-and-the-arrow-revol...

ZeroCool2u 8 months ago | parent | next [–]

You're right, something weird happened when I pasted the link in to submit it. Maybe a bug in Chrome on Android, because I just tested it again and found it only copied the link to the blog and not the specific post.

If Dang or any other mods see this, please correct the link.

yamrzou 8 months ago | root | parent | next [–]

I think it's a HN bug. I tried submitting another post from the same blog, HN software seems to rewrite the url to the homepage instead.

eloisius 8 months ago | root | parent | next [–]

I'm guessing it's because the page has this tag:

```
<link href="/blog/" rel="canonical" />
```

They're probably also screwing up their SEO this way.

bobbylarrybobby 8 months ago | root | parent | prev | next [–]

I think it's their own site. When I shared the link from my phone to my computer the same thing happened — the link got changed to the main blog page. As someone else mentioned I think it's the <link rel="canonical"> in the header.

markers 8 months ago | prev | next [–]

I highly recommend checking out polars if you're tired of pandas' confusing API and poor ergonomics. It takes some time to make the mental switch, but polars is so much more performant and has a much more consistent API (especially if you're used to SQL). I find the code is much more readable and it takes less lines of code to do things.

Just beware that polars is not as mature, so take this into consideration if choosing it for your next project. It also currently lacks some of the more advanced data operations, but you can always convert back and forth to pandas for anything special (of course paying a price for the conversion).

falcor84 8 months ago | parent | next [–]

>... of course paying a price for the conversion

As the blog mentions, once Pandas 2.0 is released, if you use Arrows types, converting between Pandas and Polars will thankfully be almost immediate (requiring only metadata adjustments).

y1zhou 8 months ago | parent | prev | next [–]

On a related note, if you come from the R community and is already familiar with the `tidyverse` suite, `tidypolars` would be a great addition.

https://github.com/markfairbanks/tidypolars

mavam 8 months ago | root | parent | next [–]

Why does this use dot-based chaining as opposed to standard dplyr piping?

hugginn 8 months ago | root | parent | next [–]

Looks like it tries to make a dplyr-like experience for python, not replace dplyr in R.

loveparade 8 months ago | prev | next [–]

There is also Polars [0], which is backed by arrow and a great alternative to pandas.

[0] https://www.pola.rs/

nerdponx 8 months ago | parent | next [–]

Polars is less ergonomic than Pandas in my experience for basic data analysis, and has less functionality for time series work. But it's better than Pandas for working with larger datasets and it's now my preferred choice for batch data processing in Python eg in ETL jobs. They both have their own places in the world.

pletnes 8 months ago | root | parent | next [–]

Yes, and with both supporting arrow, you can zero-copy share data from polars into pandas once you’ve completed your initial data load+filtering.

nerdponx 8 months ago | root | parent | next [–]

Does zero-copy work currently, or is that still theoretical/todo?

I would also be curious about Numpy, since I know you can transparently map data to a Numpy array, but that's just "raw" fixed-width binary data and not something more structured like Arrow.

ec109685 8 months ago | root | parent | next [–]

Once this is fixed: https://github.com/pola-rs/polars/pull/7084

nerdponx 8 months ago | root | parent | next [–]

I meant to ask about the other way, Polars to Pandas, but this is also useful to know.

rihegher 8 months ago | root | parent | prev | next [–]

It looks like it has already been merged, or did I miss something?

323 8 months ago | root | parent | prev | next [–]

Polars is more verbose, but simpler. It's core operations "fit the brain". I've used pandas for years, but I still have to regularly google basic indexing stuff.

brahbrah 7 months ago | root | parent | next [–]

I don’t think it’s so black and white. Depends on what you’re doing. If you’re doing lots of cross dataset operations like econometric and physical system modeling polars can get a bit too verbose. Eg how would you do the following two pandas operations in polars?

```
capacity_df - outage_df

prices.loc['2023-01'] *= 1
```

There’s many workflows and models that do thousands of these types of operations.

323 7 months ago | root | parent | next [–]

```
prices.loc['2023-01'] *= 1
```

You can always do df.to_pandas() ... prices.loc['2023-01'] *= 1 ... from_pandas() :)

More seriously, you are right, this is a tough one to do in polars. Polars seems to want to work with whole columns at a time, it doesn't give you write access to row sets.

berkle4455 8 months ago | parent | prev | next [–]

There's also SQL, when paired with optimized column-store databases that can run embedded or local (duckdb or clickhouse), you can avoid the extra hop into python libraries entirely.

eternalban 8 months ago | parent | prev | next [–]

Do you have an opinion on N5? My second hand (somewhat dated) info is that Pandas are not a good fit for unstructured (CV) data or ML.

ZeroCool2u 8 months ago | parent | prev | next [–]

Yes, Marc talks extensively about the new interop options between Pandas and Polars enabled by the pyarrow backend.

v8xi 8 months ago | prev | next [–]

Finally! I use pandas all the time particularly for handling strings (dna/aa sequences), and tuples (often nested). Some of the most annoying bugs I encounter in my code are a result of random dtype changes in pandas. Things like it auto-converting str -> np.string (which is NOT a string) during pivot operations.

There's also all types of annoying workarounds you have to do while tuples as indexes resulting from it converting to a MultiIndex. For example

srs = pd.Series({('a'):1,('b','c'):2})

is a len(2) Series. srs.loc[('b','c')] throws an error while srs.loc[('a')] and srs.loc[[('b','c')]] do not. Not to vent my frustrations, but this maybe gives an idea of why this change is important and I very much look forward to improvements in the area!

> boringg 8 months ago | parent | next [–]
>
> Oh yeah dealing with random dtype changes is a total PITA.

pama 8 months ago | prev | next [–]

Polars can do a lot of useful processing while streaming a very large dataset without ever having to load in memory much more than one row at a time. Are there any simple ways to achieve such map/reduce tasks with pandas on datasets that may vastly exceed the available RAM?

> davesque 8 months ago | parent | next [–]
>
> Not currently. But I imagine that, if Pandas does adopt Arrow in its next version, it should be able to do something like that through proper use of the Arrow API. Arrow is built with this kind of processing in mind and is continually adding more compute kernels that work this way when possible. The Dataset abstraction in Arrow allows for defining complex column "projections" that can execute in a single pass like this. Polars may be leveraging this functionality in Arrow.
>
> nerdponx 8 months ago | parent | prev | next [–]
>
> Only by writing your own routine to load and process one chunk at a time.

vegabook 8 months ago | prev | next [–]

I do a lot of...

```
some_pandas_object.values
```

to get to the raw numpy, because often dealing with raw np buffer is more efficient or ergonomic. Hopefully losing numpy foundations will not affect (the efficiency of) code which does this.

> bsdz 8 months ago | parent | next [–]
>
> Looks like they're recommending using ".to_numpy()" instead of ".values" in dev (2.*) docs:
>
> https://pandas.pydata.org/docs/dev/reference/api/pandas.Data...
>
> > nerdponx 8 months ago | root | parent | next [–]
> >
> > .values has been deprecated for a long time now, with .to_numpy() as its recommended replacement for just as long.
> >
> > > eigenvalue 8 months ago | root | parent | next [–]
> > >
> > > Wow, had no idea about this. Hopefully they can just automatically interpret .values as .to_numpy or else a ton of old code is going to break.
> > >
> > > > nerdponx 8 months ago | root | parent | next [–]
> > > >
> > > > That's what they've been doing for a long time as well!
>
> hannibalhorn 8 months ago | parent | prev | next [–]
>
> I do the same. Sounds like arrow is being implemented as an alternative, and you'll have to explicitly opt in via a dtype or a global setting, so numpy isn't going away.
>
> __mharrison__ 8 months ago | parent | prev | next [–]
>
> I often use Numba for complicated looping code for Pandas manipulation. Will be interesting to see how Arrow works with Numba.
>
> PaulHoule 8 months ago | parent | prev | next [–]
>
> … and what about interoperability with scikit-learn, PyTorch, etc?
>
> > nerdponx 8 months ago | root | parent | next [–]
> >
> > None of that has changed.

college_physics 8 months ago | prev | next [–]

My feeling is that the pandas community should be bold and consider also overhauling the API besides the internals. Maybe keep the existing API for backward compatibility but rethink what would be desirable for the next decade of pandas so to

speak. Borrowing liberally from what works in other ecosystem API's would be the idea. E.g. R, while far from beautiful can be more concise etc.

stdbrouw 8 months ago | parent | next [–]

Yeah, especially for a toolkit that is aimed at one-off analyses, I don't really see the point in API conservatism. R went through a period where many of the tools in the hadleyverse/tidyverse drastically changed from one year to the next, but the upshot is that it now has by far the nicest interface to do things like pivoting data into longer or wider formats (reshape, reshape2, tidyr), or doing operations on grouped data (plyr, dplyr, dplyr2) or data frames in general (built-in, data.table, tibble).

_Wintermute 8 months ago | root | parent | next [–]

Anecdotally, the API instability of those R packages is what stopped me using them. Try running any tidyverse code after more than 6 months, it's almost guaranteed to be broken.

If I can't rely on them in my packages or long-running projects, then I don't see the point in learning them to use in interactive work.

civilized 8 months ago | root | parent | next [–]

> Try running any tidyverse code after more than 6 months, it's almost guaranteed to be broken.

You are entitled to your opinion (which I see in every thread that discusses the tidyverse), but in my opinion this is a considerable and outdated exaggeration which will mislead the less experienced. Let's balance it out with some different perspective.

1. The core of the tidyverse, the data manipulation package dplyr, reached version 1.0 in May 2020 and made a promise to keep the API stable from that point on. To the best of my knowledge, they have done so, and anyone who wants to verify this can look at the changelogs. That's nearly 3 years of stability.

2. For several years, functions in the tidyverse have had a lifecycle indicator that appears in the documentation. It tells you if the function has reached a mature state or is still experimental. To the best of my knowledge, they have kept the promises from the lifecycle indicator.

3. I have been a full-time R and tidyverse user since dplyr was first released in 2014, and my personal experience is consistent with the two observations above. I agree with the parent commenter that the tidyverse API used to be unstable, but this was mainly in 2019 or earlier, before dplyr went to 1.0. And even back then, they were always honest about when the API might change. So now that the tidyverse maintainers are saying dplyr and other tidyverse packages are stable, I see no rational basis to doubt them.

4. Finally, even during the early unstable API period of the tidyverse, I personally did not find it such a great burden to upgrade my code as the tidyverse improved. It was actually quite thrilling to watch Hadley's vision develop and incrementally learn and use the concepts as he built them out. To use the tidyverse is to be part of something greater, part of the future, part of a new way of thinking that makes you a better data analyst.

IMHO, the functionality and ergonomics of the tidyverse are light-years ahead of any other* data frame package, and anyone who doesn't try it because of some negative anecdotes is missing out.

*No argument from me if you prefer data.table. It's got some performance advantages on large data and a different philosophy that may appeal more to some. Financial time series folks often prefer it. YMMV.

kgwgk 8 months ago | root | parent | next [–]

> To use the tidyverse is to be part of something greater, part of the future, part of a new way of thinking that makes you a better data analyst.

Wow.

_Wintermute 8 months ago | root | parent | next [–]

Yeah, I'm not even going to response to their points. I see little chance of a balanced discussion if they view an R package as a quasi-religion.

civilized 8 months ago | root | parent | next [–]

Thinking something is really great and how I want to see the future develop is not a religion or a quasi-religion. It's just my experience, one that I hope others can benefit from.

I think it's totally fine to use base R or data.table or whatever else you like. There is no one right way, and I have used all of these and more in different contexts. But if people are getting impressions of pros and cons from the discussion here on HN, they should be aware that claims of API instability are several years out of date. It would be a shame if people were scared about instability that isn't there.

323 8 months ago | parent | prev | next [–]

> consider also overhauling the API besides the internals

This never happens. Angular does not become React, React gets created instead. CoffeScript does not become TypeScript, etc.

There is too much baggage and backward compatibility that prevents such radical transformations.

Instead a new thing is created.

aflag 8 months ago | root | parent | next [–]

Which I think is a great way of doing things. Having a project that keeps breaking compatibility is very annoying. I actually think pandas changes way too much on each version. When you have millions of lines of code depending on a library, you want new features, faster speed, but you definitely don't want backwards incompatible changes.

MrMan 8 months ago | parent | prev | next [–]

the api is miserable

hopfenspergerj 8 months ago | prev | next [–]

Obviously this improves interoperability and the handling of nulls and strings. My naïve understanding is that polars columns are immutable because that makes multiprocessing faster/easier. I'm assuming pandas will not change their api to make columns immutable, so they won't be targeting multiprocessing like polars?

kylebarron 8 months ago | parent | next [–]

pandas added a copy-on-write API in 2.0, so under the hood the Arrow columns are still effectively immutable.

https://pandas.pydata.org/docs/dev/whatsnew/v2.0.0.html#copy...

ZeroCool2u 8 months ago | parent | prev | next [–]

I think if anything pandas may get additional vectorized operations, but from what I understand Polars is almost completely Rust code under the hood, which makes multiprocessing much easier compared to dealing with all the extensions and marshaling of data back and forth between Python and C/C++ that pandas does.

agolio 8 months ago | prev | next [–]

> As mentioned earlier, one of our top priorities is not breaking existing code or APIs

This is doomed then. Pandas API is already extremely bloated

Wonnk13 8 months ago | parent | next [–]

This is one of my largest pet peeves with Pandas. There's like (or was) three APIs. Half the stuff on Stack Overflow or blogs is from 2013-2015 and deprecated. I feel like I have relearn Pandas every four years since Wes launched it almost a decade ago.

__mharrison__ 8 months ago | parent | prev | next [–]

How so? Pandas is one of the most popular tools among folks doing data.

I admit that the API has issues (if/else? being the most glaring to me), notwithstanding Pandas has mass adoption because the benefits outweigh the warts.

(I happen to wish that 2.0 deprecated some of the API, but Python 3 burned a deep scar that many don't wish to relive.)

agolio 8 months ago | root | parent | next [–]

There's too many ways of doing the same thing (which I assume is already itself a relic of maintaining back-compatibility), there's inconsistencies within the API, there's "deprecated" stuff which isn't really deprecated, et cetera

```
dataframe.column
```

vs

```
dataframe['column']
```

as one example comes to mind but there is surely much more

I am of the philosophy of 'The Zen of Python'

```
There should be one-- and preferably only one --obvious way to do it.
```

Pandas is a powerful library, but when I have to use it in a workplace it usually gives me a feeling of dread, knowing I am soon to face various hacks and dataframes full of NaNs without them being handled properly, etc.

0cf8612b2e1e 8 months ago | root | parent | next [–]

Which column format would you prefer? You need the latter to address random loaded data which may contain illegal identifiers. You need the former to stifle the rumblings about Pandas verbosity.

I would get rid of the .column accessor, but you will see a lot of pushback. Notably from the R camp.

agolio 8 months ago | root | parent | next [–]

I would also get rid of the .column accessor, it has the potential to collide with function members of the DataFrame so shouldn't have been added in the first place IMO

naijaboiler 8 months ago | root | parent | prev | next [–]

Um nah. I can't imagine R follow being against .column disappearing forever. There's no equivalent in R

0cf8612b2e1e 8 months ago | root | parent | next [–]

I mean that R users would insist it remains. The R $ accessor is terse and enables fast ad hoc analyses.

naijaboiler 8 months ago | root | parent | next [–]

$ accessor does not exactly function like .column accessor. In fact, I think ['] accessor functions more like $ does.

kelipso 8 months ago | root | parent | prev | next [–]

The only reason is because it's the de facto default for python, not because of whatever cost-benefit analysis. You see the same thing with matplotlib and numpy.

nathan_compton 8 months ago | prev | next [–]

I recently switched to Polars because Pandas is so absurdly weird. Polars is much, much better. I'll be interested in seeing how Pandas 2 is.

freilanzer 8 months ago | parent | next [–]

Same, I just cannot get used to Pandas. It's weird.

nathan_compton 8 months ago | root | parent | next [–]

It is weird that its the standard thing in Python, which touts the principal of least surprise. I can't count the number of times I've been surprised by how Pandas is designed and how one is supposed to use it.

I think, in general, that Python has a lot of surprising stuff going on, but my standard for "simple" is Scheme.

freilanzer 8 months ago | root | parent | next [–]

Or SML, I think. Python in general is not as easy and simple as people make it out to be.

Wonnk13 8 months ago | prev | next [–]

So where does this place Polars? My perhaps incorrect understanding was this (Arrow integration) was a key differentiator of Polars vs Pandas.

ritchie46 8 months ago | parent | next [–]

Polars author here. Polars adheres to arrow's memory format, but is a complete vectorized query engine written in rust.

Some other key differentiatiors:

- multi-threaded: almost all operations are multi-threaded and share a single threadpool that has low contention (not multiprocessing!). Polars often is able to completely saturate all your CPU cores with useful work.

- out-of-core: polars can process datasets much larger than RAM.

- lazy: polars optimizes your queries and materializes much less data.

- completely written in rust: polars controls every performance critical operation and doesn't have to defer to third parties, this allows it to have tight control over performance and memory.

- zero-required dependencies. This greatly reduces latency. A pandas import takes >500ms, a polars import ~70/80ms.

- declarative and strict API: polars doesn't adhere to the pandas API because we think it is suboptimal for a performant OLAP library.

Polars will remain a much faster and more memory efficient alternative.

aflag 8 months ago | root | parent | next [–]

Was GPU acceleration considered? I know there's cudf which tries to offer dataframes for GPUs already. But, in my naive mind, it feels like dataframes would be a great fit for GPUs, I'm curious why there seems to be little interest in that.

jimmyechan 8 months ago | prev | next [–]

Yes! Swapping out NumPy with Arrow underneath! Excited to have the performance of Arrow the API of Pandas. Huge win for the data community!

sireat 8 months ago | prev | next [–]

The changes to handling strings and Python data types are welcome.

However I am curious on how Arrow beats NumPy on regular ints and floats.

For the last 10 years I've been under impression that int and float columns in Pandas are basically NumPy ndarrays with extra methods.

Then NumPy ndarrays are basically C arrays with well defined vector operations which are often trivially parallelizable.

So how does Arrow beat Numpy when calculating

```
mean (int64) 2.03 ms 1.11 ms 1.8x

mean (float64) 3.56 ms 1.73 ms 2.1x
```

What is the trick?

fred123 8 months ago | parent | next [–]

SIMD, multithreading, better handling of NaN, maybe other factors.

ddejohn 8 months ago | root | parent | next [–]

Numpy uses SIMD too.

sireat 8 months ago | root | parent | next [–]

Exactly why I asked, Numpy is already pretty optimized on ..ahem.. numerical operations.

Maybe some optimization on multithreading

Still I am reasonably sure Numpy utilizes multithreading already. https://in.nau.edu/arc/parallelism-in-python

And here we assume there are no NaNs.

So something else.

kylebarron 8 months ago | parent | prev | next [–]

Maybe due to SIMD?

ddejohn 8 months ago | root | parent | next [–]

Numpy uses SIMD too.

est 8 months ago | prev | next [–]

Now can mysql/pg provide binary Arrow protocols directly?

lidavidm 8 months ago | parent | next [–]

For PostgreSQL: https://github.com/apache/arrow-flight-sql-postgresql

Context: https://lists.apache.org/thread/sdxr8b0lj82zd0ql7zhk9472opq3...

Also see ADBC, which aims to provide a unified API on top of various Arrow-native and non-Arrow-native database APIs: https://arrow.apache.org/blog/2023/01/05/introducing-arrow-a...

(I am an Arrow contributor.)

Kalanos 8 months ago | prev | next [–]

if you were facing memory issues, then why not use numpy memmap? it's effing incredible https://stackoverflow.com/a/72240526/5739514

pandas is just for 2D columnar stuff; it's sugar on numpy

Gepsens 8 months ago | prev [–]

Pandas is still way too slow, if only there was an intégration with datafusion or arrow2

freilanzer 8 months ago | parent [–]

Polars is much faster, maybe that's interesting for you.

Gepsens 8 months ago | root | parent [–]

Yeah I already know about Polars, good and lightweight but datafusion is more advanced